

Lab 3

ECE 455 / ECE 555, Fall 2026

Assigned	Monday, September 21
Due	Friday, October 2, 11:59 p.m. , through Canvas

Objective

- To set up the UART
- To set up the DAC
- To explore the usage of the UART and the DAC together in a design

Teamwork

Laboratory work is done in teams of two, formed in the first week and kept for the whole semester. Only one lab report is required for each lab team. But both members must be able to explain every part of the submission to the TA. Every lab in this course is demonstrated in person to the TA during TA office hours. Bring your board, your circuit, and your results. The TA checks the operation, verifies that it meets the assigned functionality for each individual student. Both team members demonstrate, and either one may be asked to explain any part of the submission. TA office hours are posted on Canvas. Plan your demonstration in advance rather than in the last session before the due date.

Preparation

- You will need a Tiva LaunchPad and a PC with the Keil μ Vision IDE installed
- A breadboard for circuit connections
- Resistors, connecting wires, a 3.5 mm headphone jack, headphones, and two external switches

The assignment consists of three parts. The first two, the UART and the DAC, are mutually exclusive. However, in the third part, we will use both of them to create a musical piano.

This assignment uses the PLL set up at 50 MHz. Before you attempt any of the following parts, it is expected that you set up the microcontroller to operate at a 50 MHz clock frequency.

Background Setup

- Set the PLL to work at 50 MHz
- You should be familiar with using PuTTY or an equivalent tool to read the information at a serial terminal. Launch PuTTY, configure it for a serial connection with the COM port of your board, and set the baud rate to 9600.

- PuTTY is free software and can be downloaded from <https://www.chiark.greenend.org.uk/~sgtatham/putty/>.
- Submit the points labeled **R1** and **R2** as images pasted into a single PDF.

Part 1. UART Interfacing (35 points)

The UART0 is the preferred UART on the Tiva LaunchPad, as it can be connected to the PC's serial interface.

1A. Configure the UART0 at Port A (5 points)

The UART0 should be configured as 8-bit (no parity, one stop bit). The baud rate is 9600. If UART_Init() is done correctly then it should display "UART Initialization complete".

1B. Creating some UART Functions (15 points)

You should finish the following functions:

Function	Description
UART_GetChar()	Records correctly a single character input and returns its ASCII value
UART_inUDec()	Accepts input as an ASCII unsigned decimal from the keyboard and converts it into a 32-bit unsigned number
UART_OutUDec()	Outputs an unsigned decimal number to be printed on the serial terminal as an ASCII value

1C. Simple Unit Conversion (15 points)

In this part, you should create a program that displays the following choices to the user:

Choice	Description
0	Convert millimeter to centimeter
1	Convert centimeter to meter
2	Convert meter to centimeter

The program should scan the user choice, ask the user to enter the input value (the Enter key should tell the program that the input is terminated), and display the correct result as per the choice entered.

Part 2. DAC Interfacing (40 points)

2A. Set up the circuit (10 points)

You should set up:

- An R-2R DAC circuit at PB0 (LSB), PB1, and PB2 (MSB). **R1**
- An audio jack connected at the output. The best solution is to directly insert the audio jack and wrap it around some solid wires for firm placement.
- Two input switches connected at PE0 (SW1) and PE1 (SW2)

2B. Complete the following functions (20 points)

Function	Description
Inside Sound.c	
DAC_Init(void)	A function to initialize PB2 to PB0 to work as a 3-bit DAC
DAC_Out(unsigned long data)	Output 3-bit data to the DAC
Sound_Init(unsigned long period)	Input: interrupt period (will depend on which switch is pressed) for SysTick. Output: none. Set interrupt priority 1
SysTick_Handler(void)	This function is the SysTick interrupt service routine and will call the DAC_Out function to output each sample value
Inside Switch.c	
Switch_Init(void)	A function to initialize PE0 and PE1 to be used as switch inputs
Switch_In(void)	Determine which switch is pressed; should demonstrate key debounce

If the DAC initialization is done properly then you should be able to hear a 100 Hz tone in the headphones.

2C. DAC Demonstration (10 points)

The program should scan the status of the input of the switches and take the following action:

Switch status	Description
Nothing pressed	No sound at the headphones
SW1 pressed	1 kHz tone

Switch status	Description
SW2 pressed	2 kHz tone
Both switches pressed	No sound at the headphones

Note:

- No sound can simply be configured by disabling the interrupts.
- The status of the switches should remove the mechanical bounce. You can create a separate function to read the status. The function should scan the switch status, followed by a 20 ms delay and read the status of the switch again. If the switch status comes back the same (that is, pressed) then only decide in favor that the switch is pressed.

Part 3. Creating a Music Piano (25 points)

This part is the sum of two configurations: a 4-bit binary-weighted DAC at PB0 (LSB) to PB3 (MSB), and the UART to scan the input from the PC.

3A. Hardware Setup (15 points)

A 4-bit binary-weighted DAC uses resistors in a 1 / 2 / 4 / 8 resistance ratio. Select values in the 1 kΩ to 240 kΩ range. For example, you could use 1.5 kΩ, 3 kΩ, 6 kΩ, and 12 kΩ. Connect the headphone jack at the load of the binary-weighted circuit. **R2**

We will use the following inputs from the keyboard of the PC for the possible piano key inputs:

From the PC keyboard	Description
'c' or 'C'	Piano key C0 (523.251 Hz)
'd' or 'D'	Piano key D (587.330 Hz)
'e' or 'E'	Piano key E (659.255 Hz)
'g' or 'G'	Piano key G (783.991 Hz)

Check the header file Sound.h; we have already defined some symbols for you.

3B. The Piano (10 points)

In this part, you should submit a code that produces the above musical notes depending on the value passed from the PC or laptop keyboard.

- The code should scan the input from the keyboard via the UART and output the corresponding musical note to the headphones.
- Sound should be off when no key, or a key other than a valid input, is pressed.

Report

- R1 and R2 will come only in the report. You are free to add information that can help the TA in grading, however, we do not expect any extra credit for writing background theory information. This should be submitted as Report3.pdf inside the .zip file.
- The one submission report should display the names of both students and contributions of each student.
- A single Keil μ Vision project folder with all code.
- Code should be commented for better understanding and to highlight any design choices or calculations that you made.
- The code should be properly structured and should make sense. Poorly structured code, though working, may severely affect your score.
- In Embedded Systems, it is preferred that your code should be as simple as possible so that the translation into binary code is shorter in size and portable. Writing unnecessary compiler-specific directives is not a good idea.
- The report and the Keil μ Vision project folder should be submitted in a single .zip folder.

AI Use Statement

Generative AI tools are permitted on this lab, but their use must be disclosed. End your report with a short AI Use Statement naming the tool, what you used it for, and what it contributed. If you used no AI tool, write "No generative AI tools were used."

Late Work

Submissions are due at 11:59 p.m. on the stated date. **Late submissions are not accepted.** If you must miss a submission date, inform the instructor in advance, or submit a legitimate, documented reason for the absence. Failure to do so results in a grade of zero for this assignment. See the syllabus for the full policy.